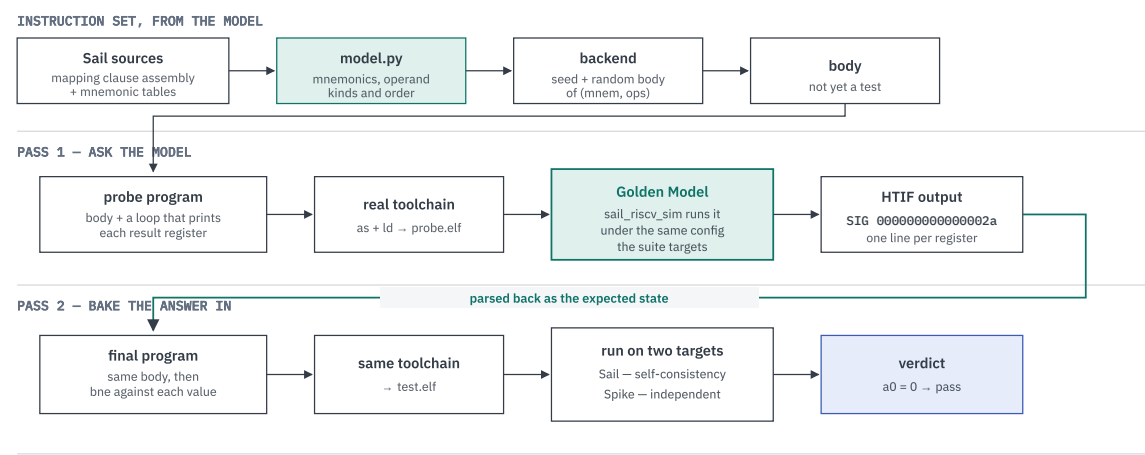


Inside the Concrete Oracle

The model is both the thing under test and the source of the answer. A probe program runs on the Golden Model, prints its own result registers, and those values are baked into a second program that asserts them.

The whole flow



Two ELF, one body. The probe and the final test contain the same instruction sequence — the probe reports what happened, the final test asserts it. Nothing in the body may depend on the link address, because the two ELF link at different sizes: that is why branches, jumps and auipc are excluded from the instruction pool.

The Sail run proves only self-consistency — the expectation came from that model. Only Spike is evidence.

The oracle never predicts anything. It observes, then asserts what it observed.

The scripts, and what each owns

FILE	OWNS
model.py	Parses the model's mapping clause assembly definitions and the mnemonic tables they reference, so mnemonics, operand kinds and operand order all come from the model. ~500 lines — the largest single piece
oracle.py	Compiles a probe, runs it on sail_riscv_sim, parses SIG <hex> back out of stdout. 51 lines — the whole oracle is this small
selfcheck.py	Turns a body into probe → oracle call → self-checking program. Owns the seeding rule and the three probe variants (integer, +FP, +vector)
program.py	Renders assembly. Owns the ACT header, the signature region and rvtest_entry_point
toolchain.py	Assembles and links exactly as the Golden Model does, reusing its crt0.S and linker script
config.py	Reads one Golden Model configuration; everything downstream derives from it
suite.py	Organises output by extension, writes the manifest
runner.py	Runs a suite on a simulator, summarises, labels which column is independent
backends/	Decides which instructions go in the body — and nothing else

A backend is deliberately tiny. Everything around it — compiling, running, organising, checking — is identical regardless of backend, so adding a generation strategy means writing one class. That is why there are seven of them.

The seven backends

BACKEND	PARSES	EXISTS BECAUSE
model	extensions/I/base_insts.sail	The base integer set — the reference implementation of the approach
model-fp	FD sources	The symbolic engine cannot execute SoftFloat
model-scalar	B and scalar-crypto sources	Bit-manipulation the symbolic path cannot reach
model-v	V sources	Vector length is runtime state the solver cannot fix
model-vk	vector-crypto sources	Needs a wider register group than plain V
model-vk64	Zvbc	vc1mul / vc1mulh have no legal SEW other than 64
template	—	Hand-written, for what neither engine reaches. Capped at 4

Three rules that make it correct

Seed every output register first

Before the body runs, every result register is set to a known constant.

```
li t0, 0x3f21c    # seed
li t1, -0x91a4    # seed
...
```

Without this, a sequence could read a register before writing it and inherit whatever `cr10` left there — which is PC-relative and therefore layout-dependent. The probe ELF and the final ELF link at different sizes, so the two runs would disagree and the test would fail for a reason that has nothing to do with the instruction under test.

Exclude anything that depends on the link address

Branches, jumps and `auipc` are pruned from the instruction pool for the same reason. This mirrors the `unsupported` denylist in the REMS generator's `generators.ml`, arrived at independently and for the same cause.

The assembler is the encoding source of truth

Nothing constructs an instruction encoding. Assembly text is rendered and handed to the real RISC-V assembler, so the bytes under test are the bytes the toolchain produces.

Together these make a parsing mistake safe. If the model parser gets an operand order wrong, the assembler rejects it or the Golden Model produces a different result — so the test is rejected or fails. It cannot silently become a wrong test that passes.

What can go wrong, and how it surfaces

FAILURE	HOW IT APPEARS
Probe traps before printing	The oracle returns zero values. One illegal instruction anywhere in the body does this, so the error message says so explicitly and the failed probe is saved for inspection
Wrong value count	Checked against the expected count before the test is built — a partial parse of the simulator output cannot become a partially-checked test
FP probe clobbering	An earlier FP probe moved results into the integer output registers before printing, overwriting them. Now all 14 values go to a memory buffer first, because there are more results than callee-saved GPRs and <code>printf</code> may clobber FP registers

How this differs from the REMS generator

`rems-project/sail-riscv-test-generation` is the closest prior art — an OCaml random instruction generator used with the `isla-testgen` flow. The approach is the same; the sourcing is not.

AREA	REMS GENERATOR	THIS FRAMEWORK
Instruction source	A generator-side instruction description, maintained alongside the generator	Parsed from the model's <code>mapping clause assembly</code> at generation time
Following the model	The description must be updated when the model's source assumptions change	An instruction added upstream is generated with no edit here; one the model does not implement cannot be generated at all
Encodings	Generator-defined	Rendered as assembly and encoded by the real toolchain
Configuration	Architecture-general	Every candidate filtered against the selected configuration's predicates first
Privileged state	Not the primary mechanism — a random stream does not place the machine in a chosen state	Built directly by the preamble builder
Fault sensitivity	A random stream is not, by itself, a test that can fail	Validation gate plus mandatory negative controls before a test joins the suite
Output format	Its own	Both the self-checking profile and the <code>riscv-arch-test</code> profile, from one format layer

The first row is the one that matters. A generator carrying its own instruction list drifts from the model the moment the model changes, and the drift is silent — a missing instruction looks exactly like one that passes. Reading the instruction set from the model's declarations is what makes “tests and coverage may change as the model evolves” a property of the design rather than a maintenance promise.

Where the two agree. The link-address denylist is the clearest case: both exclude branches, jumps and `auipc` from a no-setup random body, for exactly the same reason. Prior art was right about that, and this framework inherits the conclusion rather than rediscovering it.

Running it

```
cd autotest
PYTHONPATH=src python3 -m sailtest.cli backends

PYTHONPATH=src python3 -m sailtest.cli generate \
  --config configs/rv64.json --sail-riscv ../sail-riscv \
  --backend model-fp --count 8 --out out

PYTHONPATH=src python3 -m sailtest.cli run \
  --manifest out/rv64/manifest.json --config configs/rv64.json \
  --simulator spike
```

Generation is reproducible: the same `--seed` produces byte-identical `.S` sources. Cost is one model run per test — milliseconds, against seconds or longer for the symbolic path.

Behaviour and file sizes read from `autotest/src/sailtest/`; backend list from `sailtest.cli backends`.